LoRA4VR — Training Log

A living record of experiments, findings, and open questions

What this is. A dated log of experiments run on the LoRA4VR multi-turn RL loop. Each entry records what was tried, what happened, and how to reproduce it. Entries are added as experiments complete. Reproduction references a git commit; the code at that commit is the exact code that produced the result. The paths `docs/rung0/` and `scripts/rung0_gate_crystal.py` named in entries up to 2026-10-25 are now `docs/stage0/` and `scripts/stage0_gate_crystal.py`; a level of the experiment plan is called a stage.

System. `Qwen3-4B-Instruct-2507` with a LoRA adapter trained by branched multi-turn GRPO (`src/branched_trainer.py`). The model writes `policy(state) -> action` Python for a seeded cooling crystallizer with a hidden metastable limit (`scripts/seed_tasks/crystallizer_env.py`); the final run is scored in a sandboxed subprocess, after an optional unscored probe run on the same environment. The two Stage 0 entries dated 2026-10-24 concern the earlier slosh-tank environment, which was dropped that day. Method and experimental design: `docs/paper.tex`.

2026-10-24 Log reset: clean slate

Every entry recorded before this date was discarded, together with the environments and runs it described (CartPole and puck-on-tilted-plane training runs, Qwen2.5-Coder-3B frozen baselines, the slosh gain sweep, the slosh frozen one- and two-turn baselines, and the first branched slosh training run). None of those results is to be cited or built on. The slosh tank is the only environment going forward; the experiment ladder restarts at the environment-validity gate described in `docs/paper.tex`.

2026-10-24 Rung 0 gate: hiding the slosh sensor and the slosh length does not make the lab necessary

`b6485c0` · *hand-written controllers only, CPU; no model was run*

Setup / motivation

The ladder restarts with a validity gate: before any model is sampled, show with hand-written controllers that the scored `policy(state)` cannot succeed without information that only the turn-1 lab can provide. Two facts about the env as shipped were established first (200 seeds per value). With the actuator gain $\kappa \in \{0.65, 0.80, 1.00\}$ hidden, a fixed PD controller with the gain constant hard-coded to 0.65 succeeds in 100% of episodes at every κ; the shipped "blind" reference is the same formula with the constant at 1.0 (0 / 50.5 / 100%), so the gap it exhibited was a tuning artifact. With the slosh length L varied from 0.3 to 1.5 m and the full state observed, a PD controller that knows nothing about L also succeeds in 100% of episodes at every L: feedback on θ does the probe's job for free.

The redesign under test therefore hid the slosh state. The scored controller observed only $(t, x, v, \text{goal}, \text{steps_left})$; θ and ω were visible only to the turn-1 lab env. The hidden parameter was L (slosh period $T_n = 2\pi\sqrt{L/g}$), with κ fixed at 1. Because θ never enters the cart update in `SloshTankEnv.step`, the scored controller's inputs are independent of L, so no controller, stateful or not, can identify L in-episode; the only blind strategy is robustness. The gate measured what robustness costs. A *blind* side (one controller for every L in the set) and an *informed* side (one controller per L, the upper bound on what a perfect turn-1 measurement could buy) searched the same five controller classes with the same budget on the same 48 search seeds: shaped bang-bang moves (none/ZV/ZVD/ZVDD/EI/two-mode shapers plus a terminal PD), free-form antisymmetric open-loop profiles with 8 segments per half, pure PD on (x, v), shaped PD with memory, and feedforward plus tracking feedback. Each job ran 800 random samples and 8×150 hill-climbing steps; winners were chosen on the search seeds and reported on 200 fresh seeds per L. The gates were fixed beforehand: G1, informed $\geq 90\%$ at every L; G2, informed mean minus blind mean ≥ 40 percentage points; G3, a hand-written two-turn solution (lab measurement, then controller) $\geq 95\%$ on fresh seeds. Twelve configurations were run (1.3 CPU-hours, 7 minutes on 11 cores).

Results

L set (m)	steps	blind per L (%)	blind mean	informed per L (%)	informed mean	gap (pp)
$\{2, 3, 4\}$	80	100 / 100 / 0	67	100 / 100 / 99	100	+33
$\{2, 3, 4\}$	90	96 / 100 / 86	94	99 / 100 / 100	100	+5
$\{2, 3, 4\}$	105	100 / 100 / 100	100	99 / 100 / 100	100	0
$\{3, 4, 5\}$	90	98 / 98 / 98	98	100 / 100 / 100	100	+2
$\{3, 4, 5\}$	105, 120, 135	100 / 100 / 100	100	100 / 100 / 100	100	0
$\{2.5, 3.5, 4.5\}$	105	99 / 100 / 100	100	100 / 100 / 100	100	0
$\{2.5, 3.5, 4.5\}$	120	100 / 100 / 100	100	100 / 100 / 100	100	0
$U[2, 4]$	90	100 / 100 / 100	100	100 / 100 / 100	100	0
$U[3, 5]$	105	100 / 100 / 82	94	100 / 100 / 100	100	+6
$U[3, 5]$	120	96 / 100 / 100	99	100 / 100 / 100	100	+1

G1 passed everywhere. G2 failed in every configuration. The blind winner was always a free-form open-loop profile, typically a short bang-bang convolved with a two-frequency shaper, whose success on the continuous ranges was 100% over uniformly drawn L. G3 passed at 100% for every L in every configuration with at least 90 steps: the lab code applied a four-step pulse and read the period off the zero crossings of θ to within 0.2% (e.g. 4.008 s measured against 4.012 s true), chose the move acceleration on lab goals disjoint from the scoring goals, and the turn-2 controller was a ZV-shaped bang-bang with the two constants filled in. At 80 steps G3 fell to 90.5% at $L = 4$.

Interpretation

The premise of the redesign holds, but an unobservable parameter is not the same as a necessary measurement. Robust input shaping over ±20% of frequency uncertainty costs little time, and the optimizer found profiles that keep the residual slosh inside the tolerance for all three periods at once. The +58 pp gap seen in the pilot came from a controller family that contained only classical single-frequency shapers; once the blind side was allowed free-form profiles it vanished. A gap appears only when the budget is squeezed below what the longest period needs, and there it also hurts the informed side (G3 at 80 steps), so it is a budget artifact rather than an information gap.

Three constraints on any future design follow. First, anything the scored controller observes that depends on the hidden quantity reveals it within a step or two, because the dynamics are deterministic; the hidden quantity must affect only variables the scored controller cannot see. Second, if the hidden quantity admits a robust compromise, the lab buys nothing; frequency

uncertainty of this width admits one. Third, a scale-like parameter such as the actuator gain admits no compromise under a fixed horizon, but the env's termination on first success lets a blind open-loop plan test hypotheses in sequence: with $\kappa \in \{0.65, 0.8, 1.0\}$ a plan that stops at planned positions d, $1.25d$ and $1.54d$ succeeds under each value in turn. Success would have to be judged at the final step.

Hypotheses / next steps

1. Run the same gate on a dead-reckoning variant: the scored controller observes only $(t, \text{goal}, \text{steps_left})$, the gain (and optionally the period) is hidden, and success is evaluated at the last step with no early termination. The blind side can then be right for at most one gain value, so the gap exists by construction; the gate would confirm the informed side reaches $\geq 90\%$ and the hand-written two-turn solution $\geq 95\%$.
2. If it passes, push the two-turn solution through the real runner with the observation filter, a lab proxy that hides `env.hidden`, a lab seed disjoint from the scoring seeds, and scoring on at least four goals.
3. Check at Rung 2 that a frozen model does not already solve the lab step, since measuring a gain takes one line of code.

Open questions

Whether the no-sensor story is physically acceptable to a reviewer; whether judging success at the final step sparsifies the reward enough to hurt single-turn RL at Rung 3; whether slosh is still worth keeping once the gain carries the information gap.

Reproduce

`b6485c0`, `scripts/rung0_gate.py`: `search --n-random 800 --topk 8 --n-climb 150` over the twelve configurations listed above, and `two-turn --L 2 3 4 --steps 90`. Raw output: `docs/rung0/search_2026-10-24.log`.

2026-10-24 Rung 0 gate, second pass: dead reckoning with a hidden gain passes, provided success is judged at the final step

`3dc6581` · *hand-written controllers only, CPU; no model was run*

Setup / motivation

Following item 1 of the previous entry, the scored controller was stripped of every sensor: it observed only $(t, \text{goal}, \text{steps_left})$. The actuator gain κ was hidden (discrete $\{0.65, 0.8, 1.0\}$ or continuous $U[0.6, 1.0]$), alone with the shipped slosh length $L = 0.7\,\text{m}$ or together with a hidden L. Nothing the scored controller sees depends on either value, so the lab is the only channel, and a gain admits no robust compromise: a plan that lands on the goal under one gain lands short or long under another. One loophole remained. The shipped env ends the episode at the first step that satisfies the success test, so a blind plan can stop at the goal position implied by each gain hypothesis in turn and be credited whichever is true. A thin subclass therefore judged success at the final step only and paid the bonus only there; the shipped rule was kept as a control. The protocol was the one of the previous entry (same classes and budget on both sides, 48 search seeds, 200 fresh seeds per hidden value, gates G1–G3 fixed in advance), restricted to the open-loop classes since there is no feedback to use, plus a new multi-stop class (back-to-back shaped moves through planned positions, with an idle dwell) so the blind side could exploit the

loophole wherever the rules allowed it. Six configurations were run with the final-step rule and two as the control (126 and 24 jobs; 6 minutes on 11 cores).

Results

hidden	steps	success rule	blind mean (%)	informed mean (%)	gap (pp)	G1 / G2
$\kappa \in \{0.65, 0.8, 1\}$, $L = 0.7$	105	final step	33	100	+67	pass / pass
$\kappa \in \{0.65, 0.8, 1\}$, $L = 0.7$	90	final step	33	100	+66	pass / pass
$\kappa \sim U[0.6, 1]$, $L = 0.7$	105	final step	3 (range 18)	100	+97	pass / pass
$\kappa \in \{0.65, 0.8, 1\}$, $L \in \{2, 3, 4\}$	105	final step	33	100	+67	pass / pass
$\kappa \in \{0.65, 0.8, 1\}$, $L \in \{2, 3, 4\}$	120	final step	33	100	+66	pass / pass
$\kappa \sim U[0.6, 1]$, $L \sim U[2, 4]$	105	final step	33 (range 24)	100	+66	pass / pass
$\kappa \in \{0.65, 0.8, 1\}$, $L = 0.7$	105	first success (control)	71	99	+28	pass / FAIL
$\kappa \in \{0.65, 0.8, 1\}$, $L = 0.7$	135	first success (control)	100	100	0	pass / FAIL

Under the final-step rule the blind winner succeeded for exactly one gain value (100 / 0 / 0 across the three) in every discrete configuration and for 18% of a continuous gain range; the informed side was at 99–100% for every hidden pair, including the nine (κ, L) pairs. Under the shipped rule the blind winner was always the multi-stop class with three stops at planned positions d, $1.26d$ and $1.45d$: 13 / 100 / 100 at 105 steps and 100 / 100 / 99 at 135 steps. G3 passed at 100% for every hidden pair in all four configurations tried ($L = 0.7$ and $L \in \{2, 3, 4\}$ at 90–120 steps). The lab code recovered κ exactly from the velocity after one full-throttle step and the slosh period to within 0.3% from a four-step pulse; the turn-2 controller was the ZV-shaped bang-bang of the previous entry with the gain folded into its scaling and no terminal feedback.

Interpretation

The gap now exists by construction rather than by search: with no field sensors the scored controller cannot learn the gain, and a gain cannot be hedged, so the blind side is capped at one hypothesis (a third of a three-value set, a tolerance-width sliver of a continuous range). The measurement is trivial for the lab and useless to withhold. The control shows that the final-step rule is load-bearing and not cosmetic: given 135 steps the shipped first-success rule lets a blind plan reach 100% by visiting three candidate positions in sequence. Hiding L as well adds nothing to the gap (the gain already caps the blind side) but makes turn 2 harder and turn 1 richer, two quantities to measure in one lab episode. The continuous gain range is preferable to the discrete set: it leaves nothing to memorize and drives the blind side to a few percent.

Hypotheses / next steps

1. Env and runner changes for this variant: an observation filter so the scored `policy(state)` receives only $(t, \text{goal}, \text{steps_left})$ while the turn-1 lab env keeps the full observation; the final-step success rule with the bonus paid only at the end; L added to `HiddenParams`; a lab proxy that exposes `reset`, `step` and `observe` but not `env.hidden`; a lab seed disjoint from the scoring seeds; scoring on at least four goals.
2. Rung 0b: the hand-written two-turn solution through the real `execute_turn` path at $\geq 95\%$.
3. Rungs 1–2 as planned, with the explicit risk that a frozen model solves the one-line gain measurement from the prompt alone; if it does, the ladder tests competence rather than probing, and the paper's framing follows.

Open questions

Whether the final-step rule, which withholds the bonus until the end, sparsifies the reward enough to slow single-turn RL at Rung 3 (the shaping terms remain per step). Whether to present the

task as open-loop feedforward actuation calibrated on a bench, and whether a reviewer accepts that framing. Whether L should stay hidden or be fixed and known.

Reproduce

3dc6581, `scripts/rung0_gate.py` with the flags `--obs dead` and `--fixed-horizon`: the `search` subcommand over the six configurations in the table, written as in `"k=0.65,0.8,1.0;L=0.7@105"`, with the budget `--n-random 800`, `--topk 8`, `--n-climb 150`; the control with `--obs dead` alone; G3 with the `two-turn` subcommand and the same flags. Raw output in `docs/rung0/`: `search_dead_fixed_2026-10-24.log` and `search_dead_early_control_2026-10-24.log`.

2026-10-24 New environment: a seeded cooling crystallizer with a hidden metastable limit passes the Rung 0 gate

038e6d7 · *hand-written controllers only, CPU; no model was run*

Setup / motivation

The slosh tank was dropped after the two previous entries. Its hidden numbers were dynamics parameters, visible in the first steps of the final run, and the only variant that passed the gate did so by removing sensors. The rule that replaced it: the probe run is the same env with the same actions and the same readings as the final run, so the hidden quantity must be one that a probe run can reveal through the failure flag, the end-of-episode outcome or free repetition, and that a single scored episode cannot. A limit that is only found by crossing it is the cleanest case, because crossing it in the final run is the failure. The seeded batch cooling crystallizer (`scripts/seed_tasks/crystallizer_env.py`) implements it. A saturated solution carrying seed crystals is cooled from about 60 to about 20 °C; the cooling rate (jacket limit 1.5 °C/min, 20 s steps, 200 min horizon) is the only control; the supersaturation $\sigma = c/c^*(T) - 1$ drives seed growth ($G = k_g \sigma^{1.5}$) and is consumed by it; above the hidden metastable limit $\sigma_{\max} \sim U[0.08, 0.25]$ a shower of new nuclei ends the batch. The observation is $(t, T, c, L, T_{\text{target}}, \text{steps_left})$; $\sigma_{\max}$ appears nowhere except in the shower test, and two batches with limits 0.10 and 0.25 gave identical observations under identical actions until one showered. Success is reaching the target temperature equilibrated ($\sigma \le 0.02$); reward -0.1 per step, $+120$ on success, -80 on a shower. The fastest safe schedule rides just below the limit and accelerates as the crystals grow (the classical programmed-cooling result), so the final run needs both the limit and a real controller. The protocol was that of the previous entries: five controller families (one-step-lookahead supersaturation tracking, linear cooling, power-law cooling, piecewise-linear profile, power-law profile with a supersaturation cap), 300 random samples plus 120 hill-climbing steps from the top six, 48 search seeds, 200 fresh seeds, both sides maximizing mean env reward. Scores: success; fast success, i.e. finishing within 1.25× the time of the oracle that rides exactly at $\sigma_{\max}$; and normalized time (oracle steps over own steps, 0 on failure). Gates fixed in advance: G1 informed fast success $\ge 90\%$ at every anchor value; G2 blind fast success at least 40 pp below; G3 hand-written probe run plus final run $\ge 95\%$.

Results

hidden $\sigma_{\max}$	time wt	blind fast (%)	blind norm. time	blind reward	inf. fast (%)	inf. norm. time	inf. reward	G1 / G2 (gap)
$U[0.08, 0.25]$, anchors 0.08 / 0.165 / 0.25	0.1	7.5 (100 / 0 / 0)	0.50	91	100	1.00	91 / 108 / 110	pass / pass (+92)
$\{0.08, 0.12, 0.18, 0.25\}$	0.1	25 (100 / 0 / 0 / 0)	0.58	91	100	1.00	91 / 103 / 109 / 110	pass / pass (+75)
$U[0.08, 0.25]$ (sensitivity)	0.3	11 (0 / 0 / 0)	0.52	38	100	1.00	33 / 83 / 91	pass / pass (+89)

In both configurations the blind winner was the tracking family riding at $\sigma_t \approx 0.08$, the lowest limit in the range, which never showers and needs about 100 minutes for every batch; the informed side finished in 33 to 98 minutes depending on the limit. The open-loop families could not reach

fast success even when informed ($\leq 10\%$), so feedback on the concentration is needed to ride the limit. G3 passed at 100% fast success at every anchor in both configurations: the probe run cooled at the jacket limit until the shower, then in two more batches crept up in eighths of the remaining bracket, and recovered the limit to within 0.0002 (0.004 at the top of the range); the final run rode at 98% of the recovered value.

Interpretation

The gap comes from physics and not from a narrow controller family or a withheld sensor. A limit cannot be learned without crossing it, crossing it in the final run is the failure, and feedback on every reading is allowed. Under the env reward the blind side's best policy is caution, so the probe run buys time rather than success: two to three times faster on the easy batches, which is 12 to 19 reward points out of 120 at a time weight of 0.1. Tripling the time weight raises that to about 32 points over the range but the blind side then starts to gamble ($\sigma_t \approx 0.084$, showering the 4% of batches whose limit lies below it). Compared with the slosh tank, nothing was hidden from the final run that a plant would have, and the probe's information channel, the failure flag, is one both runs share.

Hypotheses / next steps

1. Runner: the scenario carries a `hidden` dict and the templates build the hidden parameters from it; the probe-run env is wrapped so that reset, step, observe, the success and safety tests and the config are reachable but the hidden parameters are not; the probe-run seed differs from the scoring seed (same limit, different batch); a task builder with the prompt text; tests.
2. Rung 0b: the hand-written probe run and final run through the real `execute_turn` path at $\geq 95\%$.
3. Fix the time weight (0.1 or 0.3) before any training run; it sets how much of the reward the probe is worth against how much the blind side is tempted to gamble.
4. Rungs 1–2 as planned. The probe is a few lines; a frozen model may already write it, and the ladder will say so.

Open questions

Whether 600 steps per episode is acceptable for the branched-rollout cost. Whether the growth constant k_g should also vary by batch; it is identifiable from the readings of the final run, which would show the contrast between a quantity that needs the probe run and one that does not inside a single env. Whether to add a jacket lag, which would make riding the limit harder and force a margin.

Reproduce

`038e6d7`, `scripts/rung0_gate_crystal.py`: the `search` subcommand over the two configurations `range:0.08-0.25` and `set:0.08,0.12,0.18,0.25` with the default budget (300 random, top 6, 120 climbing steps); the sensitivity row with the time weight set to 0.3 on the command line; G3 with the `two-turn` subcommand. Raw output in `docs/rung0/`, files `crystal_search`, `crystal_two_turn` and `crystal_search_timeweight03`, all dated 2026-10-24.

2026-10-25 The crystallizer replaces the slosh tank throughout the pipeline; Stage 0b passes through the runner

`d849b0f` · *hand-written probe run and final run only, CPU; no model was run*

Setup / motivation

The previous entry left the crystallizer as a standalone module next to a pipeline still wired to the slosh tank: the runner built every env with a hard-coded actuator gain, and the task files, configs, eval metric and tests were the slosh ones, so the slosh env was still needed although it had been dropped. This entry records the rewiring and the removal. A task's scenario now carries the batch seed of the final run, a different batch seed for the probe run, the hidden parameters as a dict that the runner splats into the env's `HiddenParams`, and the number of steps the time-optimal controller needs on the scoring batch. The probe run sees the env through a wrapper that forwards every public method and reading but not the hidden parameters, cannot read its own script, and runs on the probe batch. The eval metric changed from the slosh settle test to the env's own success test plus fast success (success within 1.25× the oracle time) and normalised time (oracle steps over own steps, zero on failure); fast success is the headline, since a cautious blind policy succeeds on every batch by being slow. The frozen-model script for Stages 1 and 2 was rewritten for the new task files and keeps the turn-1 mode and code per sample. The slosh env, its task files, configs, gate script, gain sweep and difficulty plan were deleted; the Stage 0 logs under `docs/rung0/` remain as the record behind the earlier entries. One inconsistency surfaced on the way: the runner calls `reset()` once after constructing the env, which under the env's seeded stream is the second batch for that seed, while the in-process helper used the first. The helper now mirrors the runner, so the stored oracle times refer to the batch the final run is scored on; the Stage 0 numbers of the previous entry were computed with the earlier helper and reproduce at their cited commit.

Stage 0b asks whether the hand-written solution survives the real execution path: the three-batch probe source from the Stage 0 gate pushed through `execute_turn` (sandbox subprocess, probe-run wrapper, probe batch), its printed bracket pasted into the final-run template, and the final policy scored with `score_sample` exactly as the trainer scores it. Pre-registered bar: fast success $\geq 95\%$. Tasks: the 24 training and 12 held-out batches plus 200 fresh batches whose limits follow the task builder's rule, 236 in all.

Results

			hand-written probe run + final run						blind cautious, same batches		
$\sigma_{\max}$ third	n	range	success	fast	norm. time	min	reward	probe err.	fast	norm. time	reward
low	78	0.080–0.136	100%	100%	0.974	0.968	99.0	0.0002	23.1%	0.674	89.7
middle	78	0.138–0.192	100%	100%	0.983	0.975	107.3	0.0002	0.0%	0.412	89.7
high	80	0.193–0.249	100%	100%	0.989	0.981	109.6	0.0002	0.0%	0.336	89.3
all	236	0.080–0.249	100%	100%	0.982	0.968	105.3	0.0002	7.6%	0.473	89.6

Gate: fast success through the real path 100% with no probe failures: pass. The 236 two-turn tasks took 1.7 s on ten workers; the env is not where the cost of a rollout lies. The full test suite (137 tests) passes, including the probe-run wrapper tests (hidden parameters unreadable, probe batch different from the scoring batch, own script unreadable) and an end-to-end test of the hand-written solution on the hardest held-out batch.

Interpretation

The numbers are the ones the in-process gate gave (probe error 2×10^{-4}, normalised time 0.97–0.99 by third), so the sandbox, the wrapper and the switch to a separate probe batch cost nothing: the pipeline carries the crystallizer end to end, and the hidden limit reaches the env only through the task file. The blind reference through the same batches is unchanged as well (fast 7.6%, normalised time 0.47), which fixes the two ends of the scale the model stages will be read against.

Hypotheses / next steps

1. Stage 1: the frozen model, one turn, no probe run; expected near the blind reference if it writes a working controller at all.
2. Stage 2: the frozen model with a probe run; read the per-sample turn-1 mode and code, not only the score, to see which probe it writes. If it already lands near the informed ceiling, the env has to get harder before any training run.
3. Fix the time weight (0.1 as shipped, or 0.3) before Stage 3; it sets how much reward the probe is worth against how much the blind side is tempted to gamble.
4. Build the stress set (limits outside the training range) that the paper's metric paragraph promises.

Open questions

Whether the probe-run wrapper is tight enough: the inner env is reachable through function introspection by a determined reader, though nothing in the reward pushes a model that way, since the honest one-batch probe already yields the limit. Whether the stress set should extend the limits downward as well, where the slowest safe batch no longer fits the 600-step horizon.

Reproduce

`d849b0f`: the task builder in `scripts/seed_tasks`, then `scripts/stage0b_two_turn.py` with 200 fresh batches and ten workers. Raw output in `docs/stage0b/`, file `two_turn_runner_2026-10-25.log`.